

Indian Institute of Technology Kharagpur

AUTUMN Semester, 2026–2027

COMPUTER SCIENCE AND ENGINEERING

Computer Organization and Architecture Laboratory

Verilog Assignment: Fast 32-bit Booth Multiplier

September 2026

Instructions:

*This is a group assignment. During today's laboratory class, every group must first prepare a hand-drawn architectural diagram of the multiplier and show it to a Teaching Assistant. The diagram must clearly separate the **Control Path** and the **Datapath** and show the major blocks and interconnections of the proposed multiplier architecture.*

*After this initial architectural discussion, each group must design and implement the complete Verilog architecture, prepare a suitable testbench, perform behavioral simulation, synthesize and implement the design, and perform a **post-implementation timing simulation**. The complete design, simulation results, and implemented timing behavior must be demonstrated to a Teaching Assistant for evaluation. An FPGA-board-based output demonstration is not required for this assignment.*

*The deadline for submission of the complete assignment will be announced separately on Moodle. The **hand-drawn architectural diagram shown to the TA during today's class must also be submitted as part of the final assignment submission.***

Question: 32-bit Signed Fast Booth Multiplier Using Carry-Save Reduction

1. Objective

Design and implement a **32-bit signed multiplier** using:

1. radix-4 / bit-pair Booth recoding;
2. parallel generation of Booth partial products;
3. a Carry-Save Adder (CSA) reduction tree; and
4. one final carry-propagating parallel adder.

The multiplier must accept two signed 32-bit two's-complement operands

`multiplicand[31:0]`

and

`multiplier[31:0]`

and generate a signed 64-bit product

`product[63:0]`.

The design must not use the Verilog multiplication operator `*` to implement the multiplier.

2. Motivation

An ordinary 32-bit binary multiplier may generate as many as

32

partial-product rows.

The fast multiplier used in this assignment improves the multiplication in two stages.

1. Booth bit-pair recoding

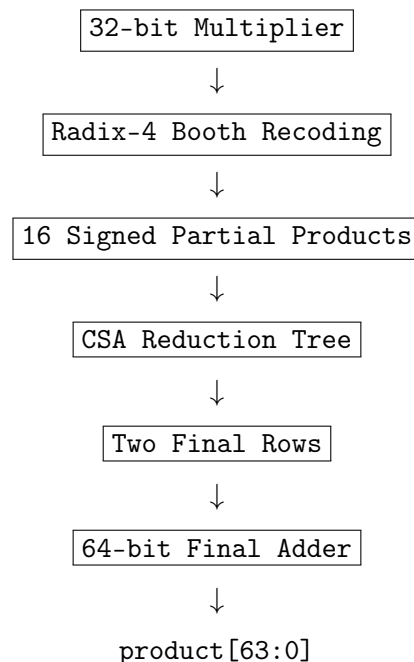
The multiplier bits are recoded in groups so that the number of partial-product rows is reduced from 32 to 16.

2. Carry-Save reduction

The 16 partial-product rows are not added sequentially using carry-propagating adders. Instead, Carry-Save Adders repeatedly reduce three rows into two rows.

Carry propagation is performed only once, at the final stage.

The overall operation is therefore:



3. Top-Level Interface

Create a top-level module named

`fast_booth_multiplier_32`

with the following interface:

Signal	Width	Description
<code>clk</code>	1 bit	System clock
<code>reset</code>	1 bit	Active-high reset
<code>start</code>	1 bit	Starts a new multiplication
<code>multiplicand</code>	32 bits	Signed two's-complement multiplicand
<code>multiplier</code>	32 bits	Signed two's-complement multiplier
<code>product</code>	64 bits	Signed two's-complement multiplication result
<code>busy</code>	1 bit	Indicates that a multiplication operation is in progress
<code>done</code>	1 bit	Indicates that the product is available

The Verilog interface should explicitly represent the operands as signed quantities, for example:

```
input wire signed [31:0] multiplicand;
input wire signed [31:0] multiplier;
output reg signed [63:0] product;
```

The most-significant bits

`multiplicand[31]`

and

`multiplier[31]`

are therefore the sign bits.

4. Radix-4 Booth Recoding

Append an additional zero bit to the right of the multiplier:

$$Q_{-1} = 0.$$

The multiplier must then be examined using overlapping groups of three bits:

$$(Q_1, Q_0, Q_{-1}),$$

$$(Q_3, Q_2, Q_1),$$

$$(Q_5, Q_4, Q_3),$$

and so on.

For the 32-bit multiplier, the final group is

$$(Q_{31}, Q_{30}, Q_{29}).$$

Thus, exactly

Booth groups are generated.

For each group, use the following radix-4 Booth recoding:

Booth Bits	Recoded Digit	Required Multiple
000	0	0
001	+1	$+M$
010	+1	$+M$
011	+2	$+2M$
100	-2	$-2M$
101	-1	$-M$
110	-1	$-M$
111	0	0

Each Booth digit therefore takes one of the values

$$d_i \in \{-2, -1, 0, +1, +2\}.$$

5. Booth Encoder Module

Create a reusable module named

`booth_encoder_radix4`

which accepts a 3-bit Booth group and generates suitable control signals for selecting one of

$$0, \quad +M, \quad +2M, \quad -M, \quad -2M.$$

Students may choose an appropriate internal encoding for the Booth digit.

For example, the encoder may generate control signals indicating:

- whether the partial product is zero;
- whether the multiplicand is multiplied by one or two; and
- whether the selected multiple is positive or negative.

The same Booth encoder design must be reused for all sixteen Booth groups.

6. Signed Partial-Product Generation

The 32-bit multiplicand must first be sign-extended to the product width.

Conceptually,

$$M_{64} = \underbrace{\{M_{31}, M_{31}, \dots, M_{31}\}}_{32 \text{ sign bits}}, M[31:0].$$

Generate the following possible values:

$$0, \quad +M_{64}, \quad +2M_{64}, \quad -M_{64}, \quad -2M_{64}.$$

Multiplication by two must be realized using a left shift:

$$2M = M \ll 1.$$

Negation must use two's-complement arithmetic.
 For Booth group i , the generated partial product is

$$PP_i = (d_i M) \ll 2i$$

where

$$i = 0, 1, \dots, 15.$$

Thus,

$$PP_0 = d_0 M,$$

$$PP_1 = d_1 M \ll 2,$$

$$PP_2 = d_2 M \ll 4,$$

and finally,

$$PP_{15} = d_{15} M \ll 30.$$

All partial products must be represented with sufficient signed width before entering the Carry-Save reduction tree.

The multiplier therefore generates

$$PP_0, PP_1, \dots, PP_{15}.$$

7. Carry-Save Adder

Create a reusable module named

`carry_save_adder_64`

having three 64-bit inputs

$$X, \quad Y, \quad Z$$

and two 64-bit outputs

$$S, \quad C.$$

At every bit position, one Full Adder must add

$$X_i, \quad Y_i, \quad Z_i.$$

The Full Adders inside the CSA must operate independently.

The carry output of bit i must **not** propagate directly to the Full Adder at bit $i + 1$.

The CSA must satisfy

$$X + Y + Z = S + (C \ll 1)$$

where the carry vector has one higher positional weight.

No horizontal carry chain must exist within a Carry-Save stage.

8. Carry-Save Reduction Tree

The sixteen Booth partial products must be reduced using a tree of Carry-Save Adders. Each Carry-Save Adder performs the reduction

$$3 \text{ rows} \longrightarrow 2 \text{ rows.}$$

The reduction must continue until exactly two rows remain.
A possible conceptual reduction is

$$16 \rightarrow 11 \rightarrow 8 \rightarrow 6 \rightarrow 4 \rightarrow 3 \rightarrow 2.$$

Students are not required to use exactly this row count at each level, but the implementation must follow a tree-style CSA reduction.

The design must **not** add all partial products using a behavioral expression such as

$$\text{pp0} + \text{pp1} + \text{pp2} + \text{pp3} + \dots + \text{pp15}$$

because such an implementation does not realize the required Carry-Save architecture.

9. Final Addition

Once the CSA tree has reduced all sixteen partial products to two rows,

$$R_0$$

and

$$R_1,$$

perform one final 64-bit carry-propagating addition:

$$\text{product} = R_0 + R_1.$$

The final addition must be implemented as a separate module named, for example,

$$\text{final_adder_64.}$$

Students may use a hierarchical fast-adder implementation already studied in the course.

The important requirement is that full carry propagation occurs only in the final addition and not at every intermediate partial-product reduction stage.

10. Worked Example: $+13 \times (-22)$

Consider the 6-bit signed example

$$M = +13 = 001101$$

and

$$Q = -22 = 101010.$$

Append

$$Q_{-1} = 0.$$

Booth Group 0

$$(Q_1, Q_0, Q_{-1}) = (1, 0, 0) = 100.$$

Therefore,

$$100 \rightarrow -2.$$

Hence,

$$PP_0 = -2M = -26.$$

Booth Group 1

$$(Q_3, Q_2, Q_1) = (1, 0, 1) = 101.$$

Therefore,

$$101 \rightarrow -1.$$

Since Booth group 1 has weight

$$2^2 = 4,$$

$$PP_1 = (-M) \ll 2 = -13 \times 4 = -52.$$

Booth Group 2

$$(Q_5, Q_4, Q_3) = (1, 0, 1) = 101.$$

Again,

$$101 \rightarrow -1.$$

Since Booth group 2 has weight

$$2^4 = 16,$$

$$PP_2 = (-M) \ll 4 = -13 \times 16 = -208.$$

Therefore,

$$P = PP_0 + PP_1 + PP_2$$

$$= -26 - 52 - 208$$

and hence

$$\boxed{P = -286.}$$

For the 32-bit implementation, the same procedure is extended to sixteen Booth groups.

11. Corresponding 32-bit Example

The 32-bit input

`multiplicand = 00000000 00000000 00000000 00001101`

represents

+13.

The 32-bit input

`multiplier = 11111111 11111111 11111111 11101010`

represents

−22.

The expected signed 64-bit product is

−286.

In hexadecimal,

<code>product = FFFFFFFF FFFEE2.</code>

12. Control Path and Datapath

The complete multiplier must be explicitly partitioned into:

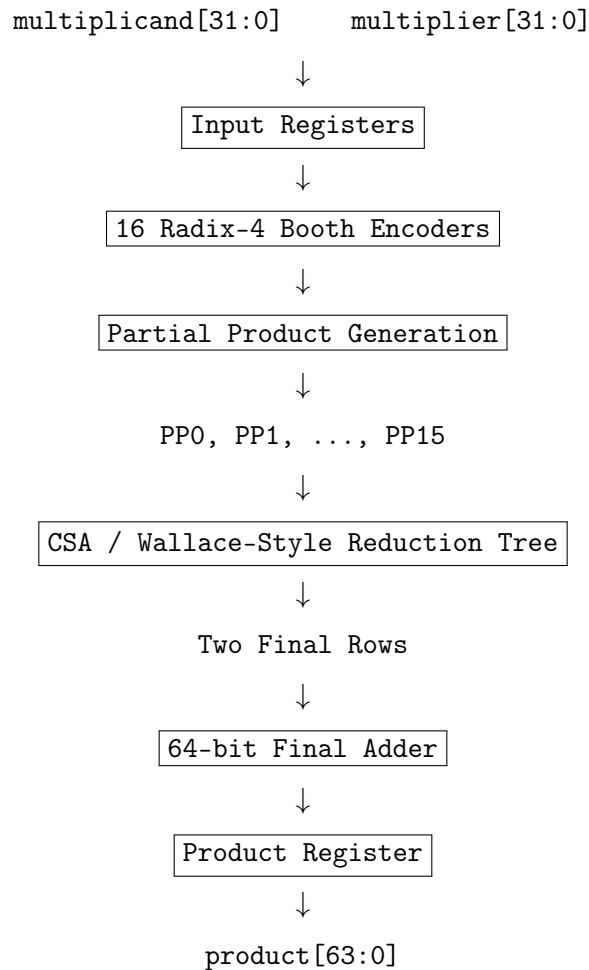
1. a **Datapath**; and
2. a **Control Path**.

Datapath

The datapath must contain the hardware responsible for arithmetic and data storage, including:

- multiplicand input register;
- multiplier input register;
- sixteen Booth encoders;
- Booth partial-product generation logic;
- CSA reduction tree;
- final 64-bit adder; and
- 64-bit product register.

A conceptual datapath is:



Control Path

Create a separate control module named

`multiplier_control`

that controls the loading and completion of the datapath.

A simple FSM may contain the states

`IDLE`, `COMPUTE`, `DONE`.

A suggested state behavior is:

State	Operation
IDLE	Wait for <code>start</code> . When <code>start</code> is asserted, capture the multiplicand and multiplier into the input registers.
COMPUTE	Allow the Booth recoding, partial-product generation, CSA reduction, and final addition to generate the multiplication result. Capture the result into the product register.
DONE	Assert <code>done</code> and hold the product stable until a new operation or reset is applied.

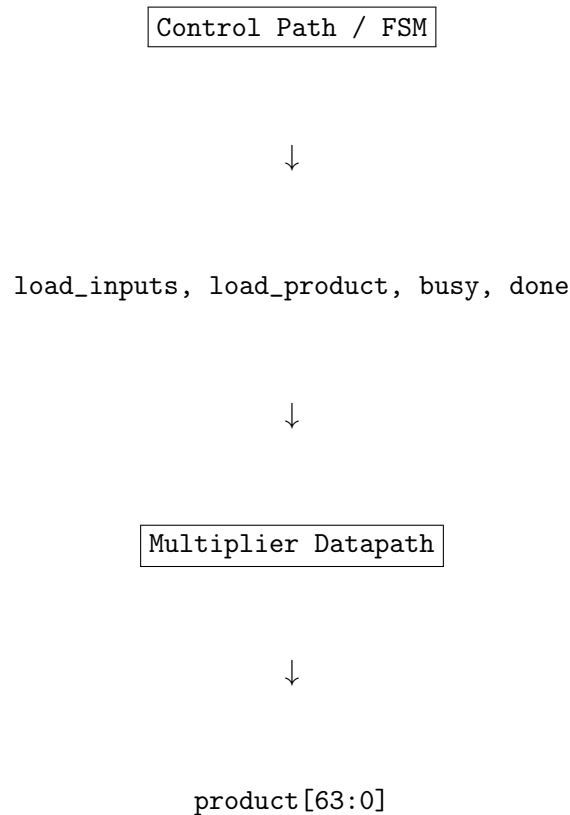
The controller may generate signals such as

`load_inputs`, `load_product`, `busy`, `done`.

The control path and datapath must be implemented as **separate Verilog modules**.

13. Control–Datapath Organization

The complete top-level architecture must contain two clearly separated parts:



The control path must not implement Booth arithmetic or partial-product reduction.
The datapath must not implement FSM state-transition logic.

14. Required Hand-Drawn Architectural Diagram

During today’s laboratory class, every group must prepare a hand-drawn architectural block diagram of the multiplier and show it to a Teaching Assistant before proceeding with the complete implementation.

The purpose of this exercise is to ensure that the multiplier architecture and the separation between the **Control Path** and **Datapath** are understood before writing the complete Verilog design.

The hand-drawn architectural diagram must clearly show:

1. the top-level multiplier;
2. the separate Control Path and Datapath;
3. multiplicand and multiplier input registers;
4. the sixteen radix-4 Booth groups;
5. Booth encoder blocks;
6. signed partial-product generation;
7. all major levels of the CSA reduction tree;

8. the two final reduced rows;
9. the final 64-bit adder;
10. the product register; and
11. all major control signals between the Control Path and Datapath.

The architectural diagram must clearly indicate the direction of

data flow

and

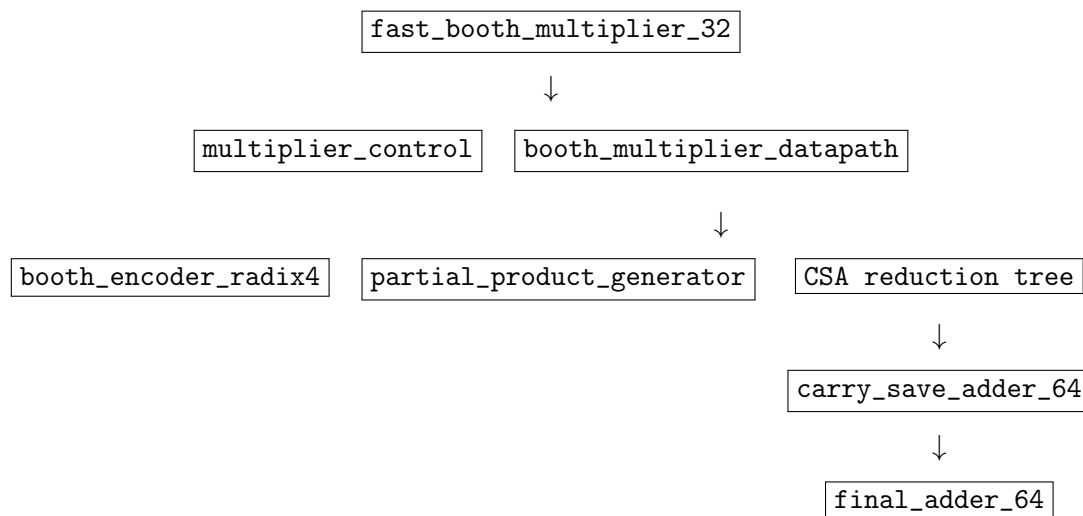
control flow.

The diagram shown to the TA in today's class must be **hand-drawn**. The TA may ask the group to explain or modify the diagram if the separation of the Control Path and Datapath, the Booth recoding, the partial-product generation, or the CSA reduction structure is not clear.

The same hand-drawn architectural diagram must be included in the final assignment submission. It may be submitted as a clear scanned copy or photograph in PDF or image format.

15. Suggested Module Hierarchy

A possible module hierarchy is:



Students may use a different internal module decomposition, provided that the required Booth, CSA, Control Path, and Datapath structures are clearly identifiable.

16. Design Restrictions

The following restrictions apply:

1. The Verilog multiplication operator `*` must not be used to implement the multiplier.
2. The sixteen Booth partial products must be explicitly generated.
3. All negative partial products must be handled using correct two's-complement signed arithmetic.
4. All partial products must be correctly sign-extended.

5. The partial products must be reduced using Carry-Save Adders.
6. A serial chain of ordinary carry-propagating additions of all sixteen partial products is not permitted.
7. Carry propagation must not occur between adjacent bit positions inside a CSA stage.
8. The final two rows must be combined using one final carry-propagating adder.
9. Control logic and datapath logic must be implemented as separate modules.
10. The implemented architecture must correspond to the submitted architectural block diagram.

The Verilog multiplication operator may be used in the **testbench only** for computing an independent reference result.

17. Behavioral Simulation

Write a self-checking testbench for

`fast_booth_multiplier_32.`

The behavioral simulation must verify:

- positive $\times$ positive;
- positive $\times$ negative;
- negative $\times$ positive;
- negative $\times$ negative;
- multiplication by zero;
- multiplication by one;
- maximum positive values;
- minimum negative values; and
- a suitable collection of random 32-bit signed operands.

At minimum, include the following test cases:

Multiplicand	Multiplier	Expected Product
13	-22	-286
100	25	2500
-100	25	-2500
-100	-25	2500
0	123456	0
1	-123456	-123456
2147483647	1	2147483647
-2147483648	1	-2147483648
-2147483648	-1	2147483648

The behavioral simulation waveform must show at least

```
        clk,    reset,    start,

multiplicand,    multiplier,    product,

        busy,    done.
```

During evaluation, students may also be asked to display selected Booth digits, partial products, CSA intermediate rows, and final reduced rows.

18. Synthesis and Implementation

After successful behavioral simulation, students must:

1. synthesize the complete multiplier;
2. perform implementation, including placement and routing;
3. inspect the synthesized and implemented design;
4. examine the resource-utilization report; and
5. examine the timing summary.

The following information must be recorded:

- LUT utilization;
- register utilization;
- inferred arithmetic resources, if any;
- critical-path delay;
- worst timing path; and
- maximum achievable clock frequency, where reported.

Students must also inspect the RTL or synthesized schematic to verify that the intended Booth and CSA architecture has been preserved.

19. Post-Implementation Timing Simulation

Post-implementation timing simulation is mandatory for this assignment.

After placement and routing, students must run the **post-implementation timing simulation** of the complete top-level design.

The same functional cases used during behavioral simulation should be applied to the implemented design.

Students must demonstrate that:

1. the implemented design produces the correct 64-bit signed product;
2. the **start**, **busy**, and **done** control behavior remains correct;
3. the product is captured correctly after the implemented combinational datapath has settled;
4. propagation delays are visible in the post-implementation waveform; and
5. the applied clock period satisfies the timing requirement of the implemented design.

The post-implementation waveform must show at least

```
        clk,    reset,    start,

multiplicand,    multiplier,    product,

        busy,    done.
```

Where practical, students should also inspect selected internal signals such as Booth recoding outputs, partial products, final CSA rows, or the input/output of the final adder.

The post-implementation simulation must be performed on the **implemented netlist**, not on the original RTL source alone.

20. Comparison of Behavioral and Post-Implementation Simulation

Students must compare the behavioral simulation with the post-implementation timing simulation.

The following points should be noted:

- Both simulations must produce the same final arithmetic result.
- Behavioral simulation represents the functional RTL behavior without physical implementation delays.
- Post-implementation timing simulation contains propagation delays introduced by the implemented FPGA resources and routing.
- Intermediate signals in the post-implementation simulation may not change simultaneously.
- The final registered product must nevertheless be captured correctly at the required clock edge.

Students must include at least one pair of waveform screenshots showing the same multiplication case in:

Behavioral Simulation

and

Post-Implementation Timing Simulation.

21. Mandatory Demonstration to the Teaching Assistant

Architecture demonstration in today's laboratory class is mandatory.

Before proceeding with the complete implementation, every group must show its **hand-drawn multiplier architecture** to a Teaching Assistant. The diagram must show a separate **Control Path** and **Datapath**, together with the major Booth-recoding, partial-product-generation, CSA-reduction, final-adder, and register blocks.

Demonstration of the completed assignment to the Teaching Assistant is also mandatory.

An FPGA-board demonstration is not required.

The demonstration must instead include:

1. the complete architectural diagram;

2. the separate Control Path and Datapath;
3. the Booth recoding architecture;
4. generation of representative partial products $0, \pm M, \pm 2M$;
5. the CSA reduction-tree architecture;
6. the final two reduced rows;
7. the final 64-bit adder;
8. behavioral simulation;
9. synthesis and implemented schematic / architecture;
10. implementation timing report; and
11. **post-implementation timing simulation.**

During the post-implementation simulation demonstration, the TA may provide arbitrary signed 32-bit operand values. Students must apply the values to their testbench and demonstrate the correct 64-bit product.

22. Submission

The deadline for submission of the complete assignment will be announced separately on **Moodle**.

Submit a single zipped folder named

`FastBoothMultiplier_{Group_Number}.zip`.

The submission must contain:

- all Verilog design source files;
- the complete self-checking testbench;
- `booth_encoder_radix4`;
- partial-product generation module(s);
- `carry_save_adder_64`;
- CSA reduction-tree module(s);
- final-adder module;
- Datapath module;
- Control Path / FSM module;
- top-level module;
- the hand-drawn architectural block diagram shown to the TA during today's laboratory class, submitted as a clear scanned copy or photograph in PDF or image format;
- behavioral simulation waveform screenshots;
- synthesis utilization report;

- implementation timing summary;
- post-implementation timing simulation waveform screenshots; and
- a short comparison of behavioral and post-implementation simulation results.

No FPGA-board output demonstration or board-output screenshot is required. Each Verilog source file must contain a header specifying:

- group number;
- names and roll numbers of all group members;
- assignment name;
- module name; and
- date of submission.